IMEN574 / IDSC574 · PROGRAMMING FOR AX

1

Overview & Logistics

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Overview & Logistics

01 **Three Ways to Program**

Write, learn, and instruct

02 **AI First, Understanding Next**

Build something, then inspect it

03 **Course Work**

Build, evaluate, and explain

01 · Three Ways to Program

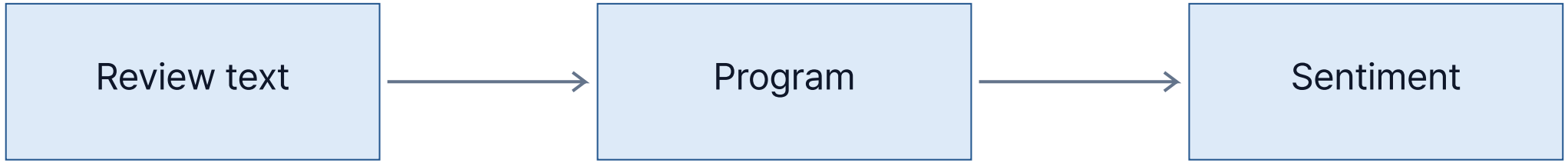
01 **Three Ways to Program**
Write, learn, and instruct

02 **AI First, Understanding Next**
Build something, then inspect it

03 **Course Work**
Build, evaluate, and explain

A Program and a Goal

The same task can be implemented in several ways



"Fast but fragile"

How is it built?

positive / negative

A fixed task exposes the differences between programming methods

Written Rules

A person specifies the decision procedure

```
def predict(text):
    return int("great" in text.lower())

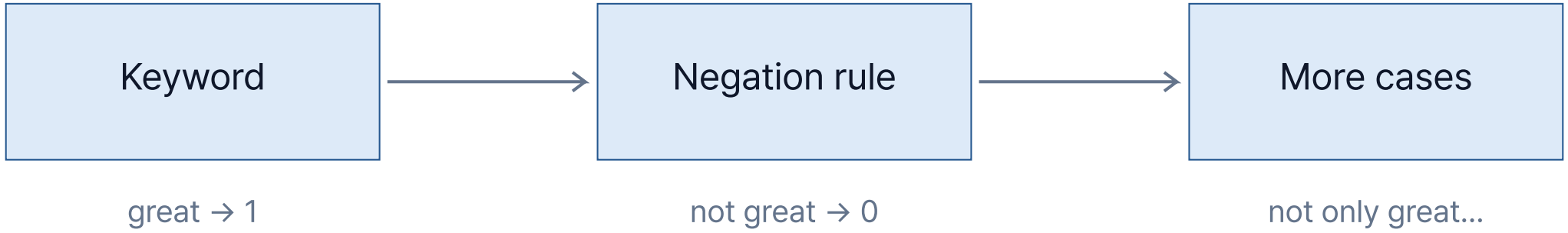
print(predict("great battery"))
print(predict("not great"))
```

```
1
1
```

The same keyword triggers both predictions

One More Rule

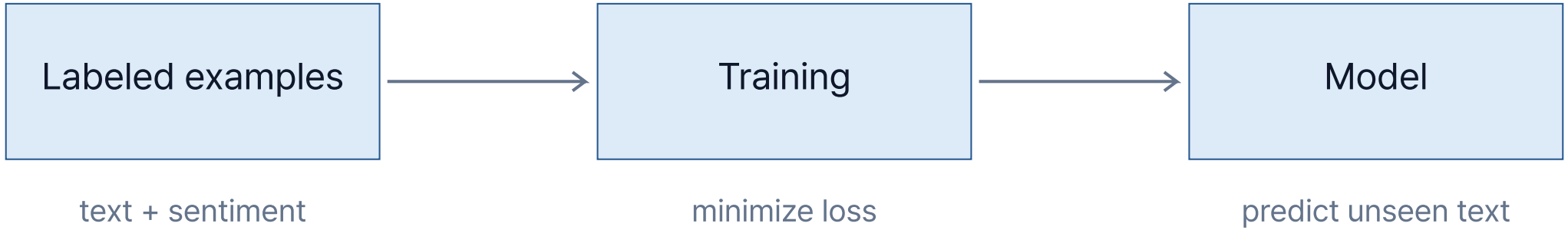
Each repair changes behavior on other inputs



Rules need an explicit treatment of new cases

Learned Parameters

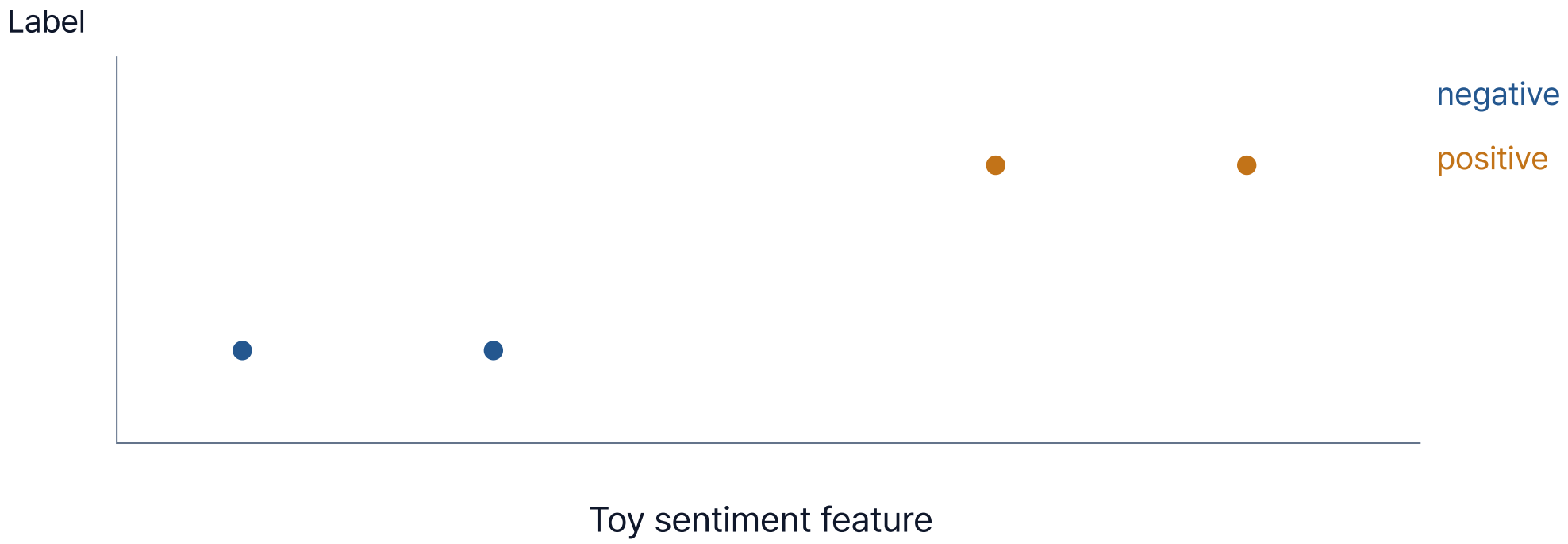
Examples determine a model's parameters



The examples replace a hand-written decision rule

A Tiny Learned Rule

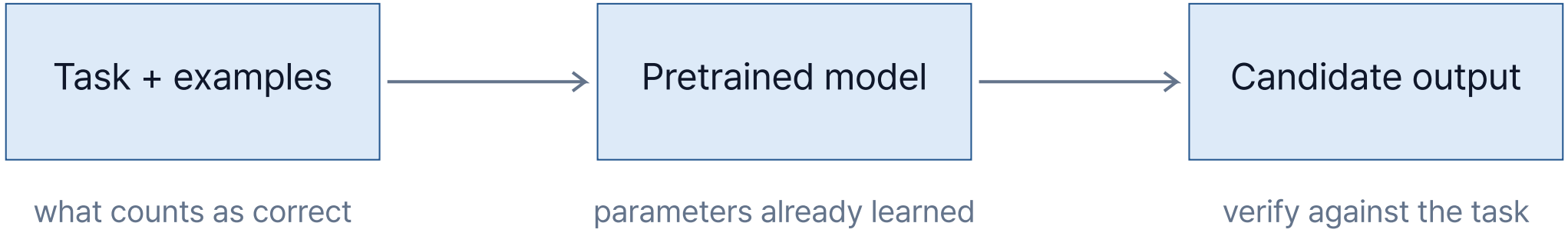
Fit a numeric decision from labeled observations



These four labeled points suggest a separating threshold

Instructions as an Interface

A pretrained model interprets task instructions



Instructions steer an already trained model

One Task, Three Artifacts

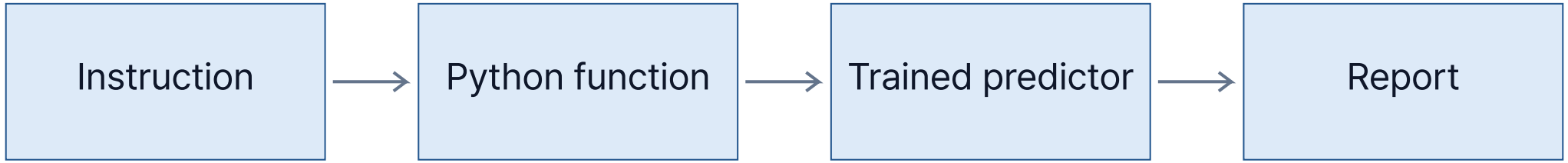
What the programmer supplies changes

Approach	Supplied artifact	Checked artifact
Written rules	Code	Execution result
Learned parameters	Data + objective	Held-out predictions
Instructions	Context + examples	Output + evidence

All three approaches still require an external check

A Mixed System

A useful workflow combines all three methods



The interfaces meet inside one practical workflow

02 · AI First, Understanding Next

01 **Three Ways to Program**

Write, learn, and instruct

02 **AI First, Understanding Next**

Build something, then inspect it

03 **Course Work**

Build, evaluate, and explain

The First Deliverable

Ask AI to create a small, testable result



one input file

one transformation

one expected format

A small deliverable makes the success condition observable

A Testable Request

State the input, output, and boundary behavior

```
Input: review_id, text
Output: review_id, predicted_label
Labels: positive, negative, uncertain
Empty text: uncertain
Keep the input order
Return code and three acceptance tests
```

Concrete boundaries make an AI-generated result inspectable

A Candidate Is Not Evidence

The generated explanation and the executed result are different artifacts



An explanation does not substitute for executing the code

The Boundary Case

Most examples can pass while the decision rule is wrong

	Score	Expected label
below	0.2	0
at boundary	0.5	1
above	0.8	1

The equality case distinguishes greater-than from greater-or-equal

Demo: An Acceptance Test

Repair the generated comparison and run the tests

The specification says score ≥ threshold; repair the comparison

```
return score > threshold ? 1 :
0;
```

score=0.2, threshold=0.5 → 0; expected 0

score=0.5, threshold=0.5 → 0; expected 1

score=0.8, threshold=0.5 → 1; expected 1

Run tests

2/3 acceptance tests passed | edit the comparison, then Run tests

A passing example alone does not check the boundary

The middle test checks the missing equality condition

Answer: The Equality Condition

A precise edit repairs the specified boundary

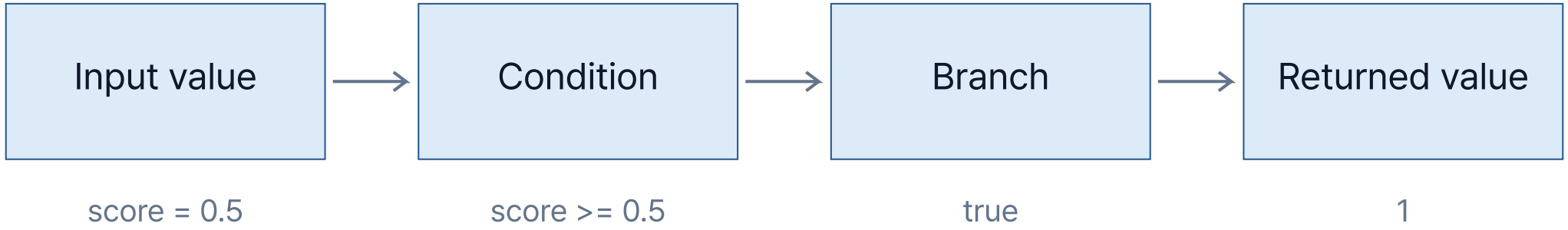
```
return score >= threshold ? 1 : 0;
```

```
score 0.2 → 0
score 0.5 → 1
score 0.8 → 1
```

The edit changes one boundary while preserving the other cases

Reading the Result

A programmer must follow values through the code



Tracing one input is enough to expose this bug

From One Script to Data

A valid function still needs a valid input table

ID	Score
A	0.2
B	missing
B	0.8

Missing and repeated rows change the meaning of a result

From Training to Evaluation

An impressive fit can fail on new observations



Illustrative error, lower is better

The training score and the held-out score answer different questions

Understanding as a Debugging Tool

Each layer exposes a different failure

Layer	Question
Python	What value reached this line?
Data	What does one row mean?
Model	What objective was optimized?
Evaluation	What remains unseen?

Understanding lets you locate the failure before asking for a repair

03 · Course Work

01 Three Ways to Program

Write, learn, and instruct

02 AI First, Understanding Next

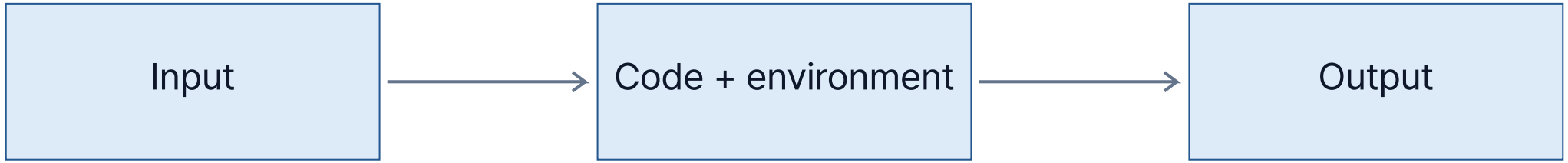
Build something, then inspect it

03 **Course Work**

Build, evaluate, and explain

A Reproducible Workspace

A result is useful when another person can rerun it



versioned fixture

recorded dependencies

saved metrics

Reproduction needs all three artifacts

An Experiment Record

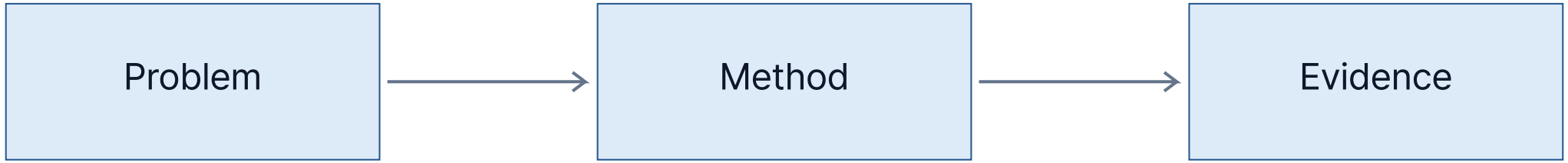
Keep the question and comparison beside the result

Run	Changed factor	Validation error
Baseline	None	0.24
Trial A	One feature	0.22
Trial B	New split	Not comparable

A changed evaluation split breaks a direct score comparison

The Team Deliverable

Explain the problem, method, and observed result



a meaningful target

a reproducible approach

an honest comparison

A useful project connects a real question to a measured result

A Baseline Before Complexity

Measure a simple method before adding a model



Illustrative validation error

Most of the improvement can come before the complex method

Honest Model Comparison

Keep the final test set outside repeated decisions



fit parameters

choose the approach

estimate unseen performance

Repeated selection belongs on validation data

Explaining a Decision

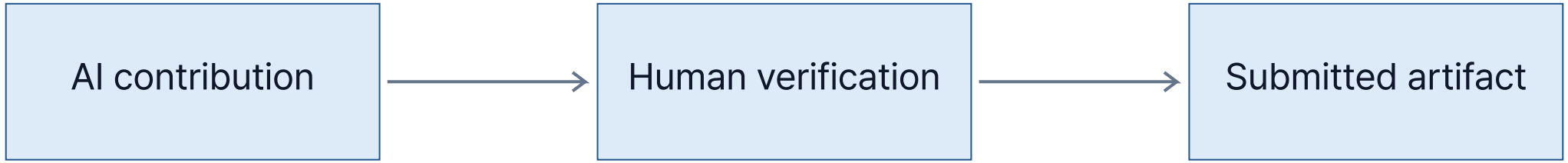
Show which evidence supports the recommendation

Claim	Evidence
The model improves	Same-split metric
The result is stable	Repeated evaluation
The run is reproducible	Code, seed, environment
The action is useful	Task-specific cost

Each claim needs the evidence that actually tests it

Responsible AI Assistance

Record the contribution and check the output



what was proposed

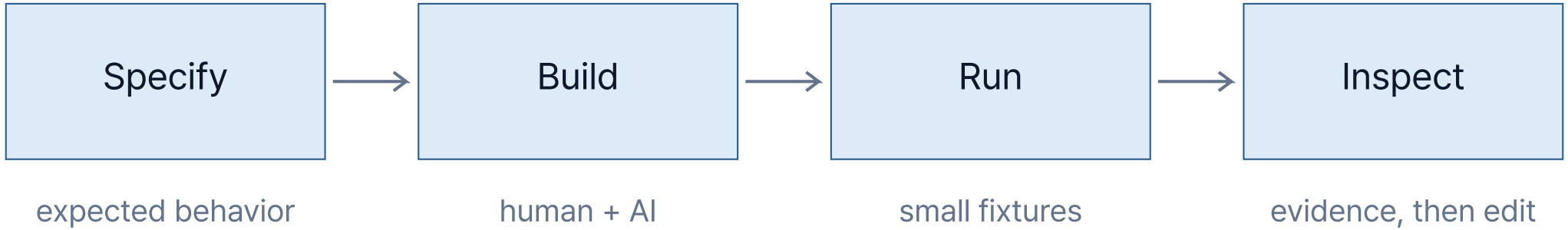
what was tested

what can be reproduced

Responsibility follows the submitted artifact

The Programming Loop

Use each failure to refine the next attempt



The loop ends with checked behavior, not a convincing explanation

Takeaway: Overview & Logistics

01 Use AI to make a first result

02 Read code to verify behavior

03 Measure models on unseen data